

ANÁLISIS DEL RETO

Juan Andrés Ospina Ramírez - J.ospinar2@uniandes.edu.co - 202614830

Jose Alexander Santos Cobos - j.santosc2@uniandes.edu.co - 202613656

Juan Esteban Lesmes Chaves - je.lesmesc1@uniandes.edu.co - 202614029

Requerimiento 1

Plantilla para el documentar y analizar cada uno de los requerimientos.

Descripción

El requerimiento nos pedía una solución con varios “ítems” dándonos el catálogo que viene siendo todos los pedidos junto con el producto que el usuario nos daba para buscar.

Primero, se filtraron todos los pedidos cargados en la `array_list`, seleccionando únicamente los que tuviesen un producto que coincidiera con el que el usuario pedía, y dejándolos en una `single_linked_list`. Ahora, sobre la lista filtrada se hace un único recorrido utilizando los `next` de cada nodo, evitando acceder por posición con `get_element` para no meterle $O(n^2)$ al algoritmo. Durante ese recorrido se acumularon las sumas, mínimos y máximos de precio, descuento, cajas enviadas e inversión en mercadeo, se contaron los pedidos por año usando un diccionario auxiliar, y se fijaron los pedidos de mayor y menor monto aplicando el criterio de desempate por menor inversión en mercadeo que era lo que se nos pedía. Por último, los resultados acumulados se vuelven promedios y se organizan en la respuesta junto con el tiempo de ejecución que se midió.

Entrada	Catalog, producto
Salidas	• Tiempo de ejecución en MS • Número total de pedidos del producto ingresado como parámetro • Promedio de <code>Price_per_Box</code>, precio más alto, precio más bajo. • Promedio de <code>Discount_Pct</code>, menor descuento y mayor descuento. • Promedio de <code>Boxes_Shipped</code>, menor y mayor número de cajas. • Promedio de <code>Marketing_Spend</code>, menor y mayor inversión en mercadeo. • Año con más pedidos (a partir de <code>Order_Date</code>). • El pedido del producto de mayor Amount, indicando su Amount respectivo. Informar también <code>Order_ID</code>, <code>Country</code>, <code>Order_Date</code> y <code>Price_per_Box</code>.

	El pedido de menor Amount dentro del producto, indicando su Amount respectivo. Informar también Order_ID, Country, Order_Date y Price_per_Box.
Implementado (Sí/No)	Sí, implementado por Juan Lesmes - 202614029

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Paso 1: Empezar el tiempo de ejecución.	O(1)
Paso 2: Indexación al catálogo de órdenes.	O(1)
Paso 3: Creación de <i>single_linked_list</i> y acceso a <i>size</i>	O(1)
Paso 4: Recorrido a cada una de las órdenes.	O(N)
Paso 5: Comparación a la variable product del registro recorrido con el producto ingresado como parámetro	O(1)
Paso 6: Inserción a la última posición de la lista SLL creada previamente	O(1)
Paso 7: Creación de la variable total que es el tamaño de la lista SLL creada previamente.	O(1)
Paso 8: Validación condicional – Caso en donde no se encontró nada.	O(1)
Paso 9: return con mensaje de “no se encontraron pedidos”.	O(1)
Paso 10: Acceso al first de la lista SLL creada usando TAD (get_element)	O(1)
Paso 11: Indexación y declaración de variables de sumas Price_Per_Box, Discount_Pct, Boxes_Shipped, Marketing_Spend junto con sus mínimos y máximos; asimismo con su pedido mayor o menor.	O(1)
Paso 12: Creación de diccionario de años	O(1)
Paso 13: División de las partes de las fechas que están separadas por guiones usando .split()	O(1)
Paso 14: Guardar en el diccionario anios la llave anios_primer que viene siendo el año del primer elemento de la SSL con su valor 1.	O(1)
Paso 15: Recorrido con while por nodos de la lista SSL creada previamente.	O(N-1) = O(N)
Paso 16: Sumas (o incrementos +=) a las llaves del “info” del nodo recorrido.	O(1)
Paso 17: Comparaciones para encontrar máximos y mínimos de cada variable dentro del “info”.	O(1)
Paso 18: Búsquedas y actualizaciones en el diccionario que está en el info de cada nodo.	O(1)
Paso 19: Avance “next” de nodos.	O(1)

Paso 20: Definición anio_mas_pedidos y conteo max	$O(1)$
Paso 21: Recorrido a cada año dentro de anios	$O(1)$
Paso 22: Comparación del valor de llave del anio evaluado con el conteo max	$O(1)$
Paso 23: Actualización de la variable anio_mas_pedido	$O(1)$
Paso 24: Declaraciones de los promedios requeridos.	$O(1)$
Paso 25: Creación de diccionarios auxiliares para la respuesta.	$O(1)$
Paso 26: Terminar tiempo de ejecución y retornar diccionario de respuesta.	$O(1)$
TOTAL	$O(N)$

Análisis

La gran mayoría de operaciones dentro de la implementación del requerimiento #1 contienen una complejidad $O(1)$, esto debido a que el requerimiento exige más que todo el uso de indexación para las búsquedas dentro de diccionarios auxiliares. Sin embargo, el algoritmo como tal usa una complejidad $O(N)$ ya que al estar implementado usando la estructura Single Linked List (SLL), el costo para acceder a la información de un nodo en la lista filtrada es más costoso que en una lista arreglo por el simple hecho de necesitar recorrer nodo por nodo en lugar de acceder a una posición con indexación y las operaciones que usan $O(1)$ están mayormente implementadas dentro de estos ciclos.

Termina siendo un buen orden de complejidad ya que, sin importar el tamaño de n , nunca será mayor que el tamaño de N , por lo tanto, el código se correrá únicamente con complejidad $O(N)$, es decir, con un comportamiento lineal. Evitando así un comportamiento cuadrático ($O(N^2)$).



Requerimiento 2

Descripción

Para resolver este requerimiento implemente el TAD ARRAY, primero use un ciclo y un condicional IF para encontrar los productos que cumplen los requisitos de entrada, luego inicialice las variables de suma para calcular los promedios que se deben devolver, luego use un ciclo para el requerimiento de encontrar el pedido más reciente dentro del rango y otro ciclo para encontrar los pedidos de mayor y menor monto, luego construí diccionarios para presentar las respuestas y se retorna lo que pide el requerimiento usando un diccionario

Entrada	Precio mínimo por caja y precio máximo por caja
Salidas	Tiempo de ejecución, Cantidad de pedidos en el rango entre precio mínimo y máximo por caja, promedio de Discount_Pct, Promedio de Marketing_Spend y Promedio de Price_per_Box para los productos dentro del filtro, pedido más reciente dentro del rango del filtro y los pedidos con mayor y menor monto
Implementado (Sí/No)	Si, implementado por Juan Ospina-202614830

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Paso 1: tomar el tiempo de inicio	$O(1)$
Paso 2: filtrar todos los datos del csv	$O(N)$
Paso 3: sumar los valores necesarios para el promedio	$O(N)$
Paso 4: sacar los promedios	$O(1)$
Paso 5: encontrar el pedido más reciente	$O(N)$
Paso 6: encontrar el pedido con mayor y menor monto	$O(N)$
Paso 7: definir diccionarios para la salida de los pedidos específicos: más reciente y de mayor y menor monto	$O(1)$
Paso 8: tomar tiempo de fin	$O(1)$
Paso 9: usar <code>delta_time()</code> para calcular el tiempo de ejecución	$O(1)$
TOTAL	$O(N)$

Análisis

La complejidad temporal total del requerimiento 2 con la solución que implemente es de $O(N)$, la operación de mayor complejidad es el filtrado, que debe hacerse para cada orden del total del csv, ya que en el resto de la función no se usan ciclos anidados ni nada de mayor complejidad, use una implementación con array que al momento de añadir `add_last()` mantiene una complejidad de $O(1)$

```

def req_2(catalog, min_price, max_price):
    """
    Retorna el resultado del requerimiento 2
    """
    start=get_time()
    orders = catalog['ordenes']
    lista_filtrada = lt.new_list()
    for i in range(lt.size(orders)):
        actual = lt.get_element(orders, i)
        if min_price <= actual['Price_per_Box'] <= max_price:
            lt.add_last(lista_filtrada, actual)
    if lt.size(lista_filtrada) == 0:
        return {
            "tiempo_ejecucion_ms": delta_time(start, get_time()),
            "Pmd_descuento": None,
            "Pmd_marketing": None,
            "Pmd_precio": None,
            "Recent_order": None,
            "Min_order": None,
            "Max_order": None,
            "total_productos": 0
        }
    else:
        descuento_total = 0
        marketing_total = 0
        precio_total = 0
        for i in range(lt.size(lista_filtrada)):
            actual = lt.get_element(lista_filtrada, i)
            descuento_total += actual["Discount_Pct"]
            marketing_total += actual["Marketing_Spend"]
            precio_total += actual["Price_per_Box"]
        pmd_descuento = descuento_total / lt.size(lista_filtrada)
        pmd_marketing = marketing_total / lt.size(lista_filtrada)
        pmd_precio = precio_total / lt.size(lista_filtrada)
        recent = lt.get_element(lista_filtrada, 0)
        for i in range(1, lt.size(lista_filtrada)):
            actual = lt.get_element(lista_filtrada, i)
            if actual["Order_Date"] > recent["Order_Date"]:
                recent = actual
            elif actual["Order_Date"] == recent["Order_Date"]:
                if actual["Amount"] > recent["Amount"]:
                    recent = actual
        min_order = lt.get_element(lista_filtrada, 0)
        max_order = lt.get_element(lista_filtrada, 0)
        for i in range(1, lt.size(lista_filtrada)):
            actual = lt.get_element(lista_filtrada, i)
            if actual["Amount"] < min_order["Amount"] or (
                actual["Amount"] == min_order["Amount"] and actual["Price_per_Box"] < min_order["Price_per_Box"]
            ):
                min_order = actual
            if actual["Amount"] > max_order["Amount"] or (actual["Amount"] == max_order["Amount"] and
                actual["Price_per_Box"] < max_order["Price_per_Box"]):
                max_order = actual
        dicreciente={"Producto": recent["Product"], "Pais": recent["Country"], "Canal": recent["Channel"], "Fecha":
            recent["Order_Date"], "Amount": recent["Amount"], "Price_per_box": recent["Price_per_Box"]}
        dicminimo={"Producto": min_order["Product"], "Pais": min_order["Country"], "Canal": min_order["Channel"],
            "Fecha": min_order["Order_Date"], "Amount": min_order["Amount"], "Price_per_box": min_order["Price_per_Box"]}
        dicmaximo={"Producto": max_order["Product"], "Pais": max_order["Country"], "Canal": max_order["Channel"],
            "Fecha": max_order["Order_Date"], "Amount": max_order["Amount"], "Price_per_box": max_order["Price_per_Box"]}
        end=get_time()
        elapsed = delta_time(start, end)
        return {
            "tiempo_ejecucion_ms": elapsed,
            "Pmd_descuento": pmd_descuento,
            "Pmd_marketing": pmd_marketing,
            "Pmd_precio": pmd_precio,
            "Recent_order": dicreciente,
            "Min_order": dicminimo,
            "Max_order": dicmaximo,
            "total_productos": lt.size(lista_filtrada)}

```

Requerimiento 3

Descripción

El requerimiento nos pide una función que nos dé promedios para pedidos de un país y un canal específicos.

La función implementada primero crea la variable start, con get_time(), que se usa para registrar el tiempo en milisegundos donde se mide la eficiencia del código, se usa orders para acceder a la llave 'ordenes' en catalog, se usa lista_filtrada importando new_list() para llamar al TAD e iniciar un array list vacío y los diccionarios productos y años para contar cuantas veces se va a repetir cada producto y año. Luego se usa un ciclo for in desde 0 hasta lt.size(orders) que es el tamaño de las órdenes. Se pide el

elemento en la posición *i* al TAD, los elementos son diccionarios con los datos del pedido, se usa un *if* para buscar el país deseado, y otro inmediatamente después para buscar el canal solicitado en el parámetro solo si el país coincidió en el *if* anterior, en ambos *if* se usa el *.lower()* y el *.strip()* para volver las palabras minúsculas y eliminar los espacios en blanco, y si ambas condiciones se cumplen el elemento se agrega a *lista_filtrada* usando la operación del TAD. Lo siguiente es verificar si la lista tiene elementos, si no, se retorna el tiempo y se indica que el total de pedidos es 0. Se saca el primer elemento filtrado de la lista para volverlo una variable, y nombra variables *suma_prices*, *suma_discount*, *suma_marketing* y *suma_boxes* para guardar el "Price_per_Box", el "Discount_Pct", el "Marketing_Spend" y el "Boxes_Shipped" del primer elemento respectivamente, se recorre con un *for in* desde 1 hasta el total de la lista, tomando el año, que son los caracteres [0:4], después se acumulan los valores en las variables *suma_prices*, *suma_discount*, *suma_marketing* y *suma_boxes* se busca *actual['product en productos']*, si se encuentra, se añade 1 al conteo, y si no, se nombra la variable en 1, esta acción se repite con el *if anio in anios*. Se calcula el promedio de las variables y dividirla en el total de pedidos. Se marca el tiempo de final y guarda todos los resultados en un *tabulate*, que es una librería que convierte datos estructurados en tablas para imprimirlo en la consola.

Entrada	El diccionario <i>Catalog</i> creado en <i>new_logic()</i> , País deseado (<i>Country</i>), Canal deseado (<i>Channel</i>)
Salidas	Tiempo de la ejecución del requerimiento en milisegundos, número total de pedidos que cumplieron el filtro, y teniendo en cuenta los pedidos que cumplen el filtro, devolver promedio de <i>Price_per_Box</i> , promedio de <i>Discount_Pct</i> , promedio de <i>Marketing_Spend</i> , promedio de <i>Boxes_Shipped</i> , producto más frecuente (<i>Product</i>) y año con más pedidos (a partir de <i>Order_Date</i>).
Implementado (Sí/No)	Si, implementado por Jose Santos - 202613656

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Paso 1: Tiempo de inicio	O(1)
Paso 2: Acceso al diccionario	O(1)
Paso 3: Crear lista filtrada	O(1)
Paso 4: Crear diccionario productos	O(1)
Paso 5: Crear diccionario anios	O(1)
Paso 6: Ciclo <i>for in</i> para filtrar información	O(N)
Paso 7: Obtener el elemento de la lista filtrada por cada iteración	O(1)
Paso 8: Suma numérica por cada iteración	O(1)
Paso 9: Comparar condiciones anidadas por cada iteración	O(1)
Paso 10: Total de datos en la lista	O(1)

Paso 11: Verificar si los datos son igual a 0	O(1)
Paso 12: Extraer el primer elemento	O(1)
Paso 13: Inicializar variable suma_prices	O(1)
Paso 14: Inicializar variable suma_discount	O(1)
Paso 15: Inicializar variable suma_marketing	O(1)
Paso 16: Inicializar variable suma_boxes	O(1)
Paso 17: ciclo for in	O(N)
Paso 18: Nombrar variable actual por cada iteración	O(1)
Paso 19: Nombrar variable anio por cada iteración	O(1)
Paso 20: suma variable suma_prices por cada iteracion	O(1)
Paso 21: suma variable suma_discount por cada iteracion	O(1)
Paso 22: suma variable suma_marketing por cada iteracion	O(1)
Paso 23: suma variable suma_boxes por cada iteracion	O(1)
Paso 24: Buscar product en productos	O(1)
Paso 25: Sumar al product o nombrarlo	O(1)
Paso 26: Buscar anio en anios	O(1)
Paso 27: Sumar al anio o nombrarlo	O(1)
Paso 28: Promediar variables	O(1)
Paso 29: ciclo for in para buscar el producto más frecuente	O(N)
Paso 30: ciclo for in para buscar el año más frecuente	O(N)
Paso 31: Terminar tiempo	O(1)
Paso 32: Return tabulate	O(1)
TOTAL	O(N)

Análisis

La complejidad es de $O(N)$, esto gracias a los ciclos for in donde se recorre n veces según los elementos que se tengan que iterar de la lista filtrada. Dentro de los ciclos no se utilizan líneas con mayor complejidad, entonces se sabe que en cada for in se debe ejecutar si o si n veces.


```
def req_3(catalog, country, channel):
    """
    Retorna el resultado del requerimiento 3
    """
    start = get_time()
    orders = catalog['ordenes']
    lista_filtrada = lt.new_list()

    for i in range(lt.size(orders)):
        actual = lt.get_element(orders, i)
        if actual['Country'].lower().strip() == country.lower().strip():
            if actual['Channel'].lower().strip() == channel.lower().strip():
                lt.add_last(lista_filtrada, actual)

    total = lt.size(lista_filtrada)

    if total == 0:
        return {
            "tiempo_ejecucion_ms": delta_time(start, get_time()),
            "Total_pedidos": 0,
        }

    primero = lt.get_element(lista_filtrada, 0)
    suma_price = primero['Price_per_Box']
    suma_discount = primero['Discount_pct']
    suma_marketing = primero['Marketing_Spend']
    suma_boxes = primero['Boxes_Shipped']
    pedido_mayor = primero
    pedido_menor = primero

    for i in range(1, total):
        actual = lt.get_element(lista_filtrada, i)

        suma_price += actual['Price_per_Box']
        suma_discount += actual['Discount_pct']
        suma_marketing += actual['Marketing_Spend']
        suma_boxes += actual['Boxes_Shipped']

        if actual['Amount'] > pedido_mayor['Amount'] or (
            actual['Amount'] == pedido_mayor['Amount'] and actual['Price_per_Box'] < pedido_mayor['Price_per_Box']):
            pedido_mayor = actual

        if actual['Amount'] < pedido_menor['Amount'] or (
            actual['Amount'] == pedido_menor['Amount'] and actual['Price_per_Box'] < pedido_menor['Price_per_Box']):
            pedido_menor = actual

    pmd_price = suma_price / total
    pmd_discount = suma_discount / total
    pmd_marketing = suma_marketing / total
    pmd_boxes = suma_boxes / total

    dic_mayor = {
        "Order_ID": pedido_mayor['Order_ID'],
        "Product": pedido_mayor['Product'],
        "Order_Date": pedido_mayor['Order_Date'],
        "Price_per_Box": pedido_mayor['Price_per_Box'],
        "Amount": pedido_mayor['Amount']
    }

    dic_menor = {
        "Order_ID": pedido_menor['Order_ID'],
        "Product": pedido_menor['Product'],
        "Order_Date": pedido_menor['Order_Date'],
        "Price_per_Box": pedido_menor['Price_per_Box'],
        "Amount": pedido_menor['Amount']
    }

    end = get_time()

    return {
        "tiempo_ejecucion_ms": delta_time(start, end),
        "Total_pedidos": total,
        "Pmd_price_per_box": pmd_price,
        "Pmd_discount_pct": pmd_discount,
        "Pmd_marketing_spend": pmd_marketing,
        "Pmd_boxes_shipped": pmd_boxes,
        "Pedido_mayor_amount": dic_mayor,
        "Pedido_menor_amount": dic_menor
    }
```

Requerimiento 4

Descripción

Breve descripción de como abordaron la implementación del requerimiento

Entrada	Recibe el catalogo con todos los datos, el producto del que se buscan pedidos y el país del que se buscan los pedidos
Salidas	Devuelve el tiempo de ejecución en ms, el total de pedidos que cumplen las condiciones, promedios de Price_per_box, Discount_pct, Marketing_spend y de Boxes_shipped para los pedidos que cumplen las condiciones y los dos pedidos de mayor monto dentro de las condiciones
Implementado (Sí/No)	Si, se implemento

Análisis de complejidad

Pasos	Complejidad
Paso 1: tomar tiempo de inicio	$O(1)$
Paso 2: crear la lista filtrada y filtrar los pedidos	$O(N)$
Paso 3: verificar que allá pedidos que cumplan las condiciones	$O(1)$
Paso 4: crear variables de suma para los promedios	$O(1)$
Paso 5: encontrar los dos pedidos con mayor monto y sumar los valores para los promedios	$O(N)$
Paso 6: calcular los promedios requeridos	$O(1)$
Paso 7: definir diccionarios con los datos requeridos para los dos pedidos de mayor monto	$O(1)$
Paso 8: tomar tiempo de fin	$O(1)$
Paso 9: calcular tiempo de ejecución	$O(1)$
TOTAL	$O(N)$

Análisis

La complejidad del algoritmo para el requerimiento 4 es $O(N)$, la mayor complejidad es definida por el número de elementos en el CSV con todos los pedidos, porque se deben revisar uno por uno para filtrarlos, después de eso los ciclos que se usan para sumar en los promedios y en encontrar los dos pedidos de mayor monto, que en el peor de los casos, ósea si todos los elementos del CSV cumplen las condiciones, tienen complejidad $O(N)$

```

def req_4(catalog,product,country):
    """
    Retorna el resultado del requerimiento 4
    """
    start = get_time()
    orders = catalog['ordenes']
    lista_filtrada = lt.new_list()
    suma_price_per_box = 0
    suma_discount_pct = 0
    suma_marketing_spend = 0
    suma_boxes_shipped = 0
    for i in range(lt.size(orders)):
        actual = lt.get_element(orders, i)
        if actual["Product"].lower().strip() == product.lower().strip() and actual["Country"].lower().strip() == country.lower().strip():
            lt.add_last(lista_filtrada, actual)
        if lt.size(lista_filtrada) == 0:
            return {"Tiempo_ejecucion_ms": delta_time(start, get_time()), "message": "No hay pedidos que cumplan el filtro"}
    max1 = lt.get_element(lista_filtrada, 0)
    max2 = None
    suma_price_per_box = max1["Price_per_Box"]
    suma_discount_pct = max1["Discount_Pct"]
    suma_marketing_spend = max1["Marketing_Spend"]
    suma_boxes_shipped = max1["Boxes_Shipped"]
    for i in range(1, lt.size(lista_filtrada)):
        actual = lt.get_element(lista_filtrada, i)
        suma_price_per_box += actual["Price_per_Box"]
        suma_discount_pct += actual["Discount_Pct"]
        suma_marketing_spend += actual["Marketing_Spend"]
        suma_boxes_shipped += actual["Boxes_Shipped"]
        if actual["Amount"] != max1["Amount"]:
            if actual["Amount"] > max1["Amount"]:
                max2 = max1
                max1 = actual
            elif actual["Amount"] == max1["Amount"]:
                if actual["Marketing_Spend"] < max1["Marketing_Spend"]:
                    max2 = max1
                    max1 = actual
                elif actual["Marketing_Spend"] == max1["Marketing_Spend"]:
                    if actual["Order_ID"] < max1["Order_ID"]:
                        max2 = max1
                        max1 = actual
            elif max2 is None or actual["Amount"] > max2["Amount"]:
                max2 = actual
            elif actual["Amount"] == max2["Amount"]:
                if actual["Marketing_Spend"] < max2["Marketing_Spend"]:
                    max2 = actual
                elif actual["Marketing_Spend"] == max2["Marketing_Spend"]:
                    if actual["Order_ID"] < max2["Order_ID"]:
                        max2 = actual
    pmd_price_per_box = suma_price_per_box / lt.size(lista_filtrada)
    pmd_discount_pct = suma_discount_pct / lt.size(lista_filtrada)
    pmd_marketing_spend = suma_marketing_spend / lt.size(lista_filtrada)
    pmd_boxes_shipped = suma_boxes_shipped / lt.size(lista_filtrada)
    dict1={"Order_ID": max1["Order_ID"],"Canal": max1["Channel"],"Fecha": max1["Order_Date"],"Cajas_enviadas":
    max1["Boxes_Shipped"],"Monto": max1["Amount"]}
    if max2 is not None:
        dict2={"Order_ID": max2["Order_ID"],"Canal": max2["Channel"],"Fecha": max2["Order_Date"],"Cajas_enviadas":
    max2["Boxes_Shipped"],"Monto": max2["Amount"]}
    else:
        dict2= None
    end= get_time()
    elapsed= delta_time(start, end)

    return {"Tiempo_ejecucion_ms": elapsed,
            "Pmd_price_per_box": pmd_price_per_box,
            "Pmd_discount_pct": pmd_discount_pct,
            "Pmd_marketing_spend": pmd_marketing_spend,
            "Pmd_boxes_shipped": pmd_boxes_shipped,
            "Max1": dict1,
            "Max2": dict2,
            "Total_pedidos": lt.size(lista_filtrada)}

```

Requerimiento 5

Plantilla para el documentar y analizar cada uno de los requerimientos.

Descripción

Primero, el código va recorriendo uno a uno todos los pedidos del catálogo y guarda en una nueva lista únicamente aquellos que corresponden al producto del que se está buscando su información y que, además, están dentro de las fechas dadas. Después, revisa esa lista filtrada para sumar los valores de cada uno de ellos e ir comparando uno a uno los pedidos hasta encontrar el mayor o el menor de todos, aplicando las reglas de desempate en caso de que haya pedidos con el mismo monto. En la parte final,

calcula los promedios de la muestra, organiza los datos del pedido ganador, y presenta el resumen completo en una tabla.

Entrada	Se recibe el catálogo con todos los pedidos, una palabra de filtro que puede ser MAYOR o MENOR, un producto, una fecha de inicio y una fecha de final
Salidas	Se devuelve con la función tabulate: el tiempo de la ejecución del requerimiento en milisegundos, filtro de selección del monto MENOR o MAYOR, número total de pedidos que cumplieron el filtro, Price_per_Box, Boxes_Shipped, Amount, Channel, Order_Date, Marketing_Spend, Price_per_Box, Boxes_Shipped, y Marketing_Spend.
Implementado (Sí/No)	Si, se implemento

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Paso 1: Ciclo de n iteraciones	$O(N)$
Paso 2: Verificar condicional	$O(1)$
Paso 3: Búsqueda y desempate	$O(N)$
Paso 4: Diccionario y return	$O(1)$
TOTAL	$O(N)$

Análisis

Como se debe iterar todos los elementos en el for in, la complejidad mayor que se presenta en el algoritmo que termina siendo la complejidad final es $O(N)$, en ese proceso no hay complejidades mayores dentro de la iteración, ya que dentro todas se ejecutan $O(1)$ veces.

```
def req_5(catalog, filtro, producto, fecha_inicial, fecha_final):
    """
    Retorna el resultado del requerimiento 5
    """
    start = get_time()
    orders = catalog['ordenes']
    lista_filtrada = lt.new_list()
    for i in range(lt.size(orders)):
        actual = lt.get_element(orders, i)
        if (actual['Product'].lower() == producto.lower()) and (fecha_inicial <= actual['Order_Date'] <= fecha_final):
            lt.add_last(lista_filtrada, actual)
    if lt.size(lista_filtrada) == 0:
        return {
            "tiempo_ejecucion_ms": delta_time(start, get_time()),
            "Total_pedidos": 0,
            "Mensaje": "No se encontraron pedidos en el rango de fechas"
        }

    suma_price_per_box = 0
    suma_boxes_shipped = 0
    suma_marketing_spend = 0
    actual = lt.get_element(lista_filtrada, 0)
    respuesta = actual
    for i in range(lt.size(lista_filtrada)):
        actual = lt.get_element(lista_filtrada, i)

        suma_price_per_box += actual["Price_per_Box"]
        suma_boxes_shipped += actual["Boxes Shipped"]
        suma_marketing_spend += actual["Marketing_Spend"]
        if filtro == "MAVON":

            if actual["Amount"] > respuesta["Amount"]:
                respuesta = actual
            elif (actual["Amount"] == respuesta["Amount"]):
                if actual["Price_per_Box"] < respuesta["Price_per_Box"]:
                    respuesta = actual
                elif (actual["Price_per_Box"] == respuesta["Price_per_Box"]):
                    if actual["Marketing_Spend"] < respuesta["Marketing_Spend"]:
                        respuesta = actual
            elif filtro == "MENDOR":

                if actual["Amount"] < respuesta["Amount"]:
                    respuesta = actual
                elif (actual["Amount"] == respuesta["Amount"]):
                    if actual["Price_per_Box"] < respuesta["Price_per_Box"]:
                        respuesta = actual
                    elif (actual["Price_per_Box"] == respuesta["Price_per_Box"]):
                        if actual["Marketing_Spend"] < respuesta["Marketing_Spend"]:
                            respuesta = actual
        pmd_price_per_box = suma_price_per_box / lt.size(lista_filtrada)
        pmd_boxes_shipped = suma_boxes_shipped / lt.size(lista_filtrada)
        pmd_marketing_spend = suma_marketing_spend / lt.size(lista_filtrada)
        dic_respuesta = {"Price_per_box": respuesta["Price_per_Box"], "Cajas_enviadas": respuesta["Boxes Shipped"],
            "Canal": respuesta["Channel"], "Fecha": respuesta["Order_Date"], "Monto": respuesta["Amount"], "Marketing_Spend":
            respuesta["Marketing_Spend"]}
    end = get_time()
    elapsed = delta_time(start, end)
    return {"tiempo_ejecucion_ms": elapsed,
        "Filtro": filtro,
        "Pmd_price_per_box": pmd_price_per_box,
        "Pmd_marketing_spend": pmd_marketing_spend,
        "Pmd_boxes_shipped": pmd_boxes_shipped,
        "Total_pedidos": lt.size(lista_filtrada),
        "respuesta": dic_respuesta
    }
```

Requerimiento 6

Descripción

Entrada	Fecha inicial, fecha final y el catalogo
Salidas	-Tiempo de ejecución. -Número de pedidos dentro del rango de fechas. -Nombre del canal más usado: su total de pedidos que cumplen el rango y su total de recaudo. -Nombre del canal que más recauda, su total de pedidos que cumplen el rango y su total de recargo. -Para cada canal del filtro devuelve también: -promedio de “Price_per_box” -promedio de “Marketing_spend” -su pedido más costoso y el más barato
Implementado (Sí/No)	Si, se implemento

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Paso 1: tomar tiempo de inicio	$O(1)$
Paso 2: crear diccionario para los canales	$O(1)$
Paso 3: crear la lista filtrada, filtrar los datos del CSV y llenar el diccionario canales con los datos filtrados	$O(N)$
Paso 4: revisa que la lista filtrada no este vacía	$O(1)$
Paso 5: encuentra los 2 canales específicos que nos piden y crea los diccionarios para cada canal	$O(N)$
Paso 6: tomar tiempo final	$O(1)$
Paso 7: retorno	$O(1)$
TOTAL	$O(N)$

Análisis

La complejidad de este requerimiento es $O(N)$, porque el proceso de mayor complejidad es la revisión de cada pedido con el filtro, el resto de operaciones en su mayoría tienen complejidad constante, excepto por la búsqueda y los diccionarios dentro del diccionario de canales, que tiene complejidad $O(N)$ en el peor de los casos, que sería si cada pedido tiene un canal diferente, porque el código se repite una vez por cada canal encontrado entre los productos del filtro

```

def req_6(catalog, fecha_inicial, fecha_final):
    """
    Retorna el resultado del requerimiento 6
    """
    start = get_time()
    orders = catalog['ordenes']
    canales = {}
    lista_filtrada = sll.new_list()
    for i in range(sll.size(orders)):
        actual = lt.get_element(orders, i)
        order_date = actual["Order_Date"]

        if fecha_inicial <= order_date <= fecha_final:
            sll.add_last(lista_filtrada, actual)

            if actual["Channel"] not in canales:
                canales[actual["Channel"]] = {"total_amount": 0, "total_price_per_box": 0, "total_marketing_spend":
0, "max_order": None, "min_order": None, "total_orders": 0}
            canales[actual["Channel"]]["total_amount"] += actual["Amount"]
            canales[actual["Channel"]]["total_price_per_box"] += actual["Price_per_Box"]
            canales[actual["Channel"]]["total_marketing_spend"] += actual["Marketing_Spend"]
            canales[actual["Channel"]]["total_orders"] += 1
            if canales[actual["Channel"]]["max_order"] is None or actual["Amount"] > canales[actual["Channel"]][
"max_order"]["Amount"]:
                canales[actual["Channel"]]["max_order"] = actual
            if canales[actual["Channel"]]["min_order"] is None or actual["Amount"] < canales[actual["Channel"]][
"min_order"]["Amount"]:
                canales[actual["Channel"]]["min_order"] = actual
            if sll.size(lista_filtrada) == 0:

                return {
                    "Tiempo_ejecucion_ms": delta_time(start, get_time()),
                    "Total_pedidos": 0,
                    "Canal_mas_usado": None,
                    "Canal_mas_recauda": None,
                    "Reporte_por_canal": {}
                }

            canal_mas_usado = None
            max_pedidos = 0
            canal_mayor_amount = None
            max_amount = 0

            for ch, datos in canales.items():
                if datos["total_orders"] > max_pedidos:
                    max_pedidos = datos["total_orders"]
                    canal_mas_usado = ch
                if datos["total_amount"] > max_amount:
                    max_amount = datos["total_amount"]
                    canal_mayor_amount = ch
            reporte_canales = {}
            for ch, datos in canales.items():
                max_ord = datos["max_order"]
                min_ord = datos["min_order"]
                reporte_canales[ch] = {
                    "Promedio_Price_per_Box": datos["total_price_per_box"] / datos["total_orders"],
                    "Promedio_Marketing_Spend": datos["total_marketing_spend"] / datos["total_orders"],
                    "Pedido_mas_costoso": {
                        "Order_ID": max_ord["Order_ID"],
                        "Product": max_ord["Product"],
                        "Country": max_ord["Country"],
                        "Order_Date": max_ord["Order_Date"],
                        "Boxes_Shipped": max_ord["Boxes_Shipped"],
                        "Amount": max_ord["Amount"]
                    },
                    "Pedido_mas_barato": {
                        "Order_ID": min_ord["Order_ID"],
                        "Product": min_ord["Product"],
                        "Country": min_ord["Country"],
                        "Order_Date": min_ord["Order_Date"],
                        "Boxes_Shipped": min_ord["Boxes_Shipped"],
                        "Amount": min_ord["Amount"]
                    }
                }
            end = get_time()
            elapsed = delta_time(start, end)
            return {
                "Tiempo_ejecucion_ms": elapsed,
                "Total_pedidos": sll.size(lista_filtrada),
                "Canal_mas_usado": {
                    "Nombre": canal_mas_usado,
                    "Total_pedidos": canales[canal_mas_usado]["total_orders"],
                    "Total_recaudo": canales[canal_mas_usado]["total_amount"]
                },
                "Canal_mas_recauda": {
                    "Nombre": canal_mayor_amount,
                    "Total_pedidos": canales[canal_mayor_amount]["total_orders"],
                    "Total_recaudo": canales[canal_mayor_amount]["total_amount"]
                },
                "Reporte_por_canal": reporte_canales
            }

```